

版本: v0.4 Draft
日期: 2026-02-04
性質: 架構改善計畫 (--plan 模式輸出)
研究範疇: 網路搜尋整合 × 文字排版精準化 × 圖片搜尋優化

目錄

- 1. 執行摘要
- 2. 現況分析與痛點
- 3. 改善計畫一：網路搜尋 + LLM Prompt 整合
- 4. 改善計畫二：精準美觀文字排版
- 5. 改善計畫三：PEXELS API 精準圖片搜尋
- 6. v0.4 管線全景圖
- 7. 實作優先序與分期規劃
- 8. 風險評估與緩解策略
- 9. 附錄：技術細節與參考

1. 執行摘要

目標

將 TeacherAssist V2 從「LLM 純生成」升級為「搜尋增強 + 精準排版 + 智慧配圖」的專業簡報系統。

三大改善軸線

#	軸線	核心價值	影響範圍
1	網路搜尋 + LLM Prompt 整合	內容品質從「LLM 幻想」→「有據可查」	Stage 0.5 (新增) + Stage 2 (強化)
2	精準美觀文字排版	排版從「塞文字」→「專業版面」	Stage 4 + AutoFitter (重寫)
3	PEXELS 精準圖片搜尋	配圖從「隨機找圖」→「語意匹配」	Stage 3 (重構)

預期 v0.4 管線

- Stage 0 : InputClassifier — 輸入分類
- Stage 0.5: WebSearcher [NEW] — 網路搜尋增強 (僅 SEARCH 模式)
- Stage 1 : TemplateAnalyzer — 模板分析
- Stage 2 : ContentGenerator — LLM 內容生成 (注入搜尋結果)
- Stage 2.5: BatchKeywordGen [NEW] — 批次圖片關鍵字生成

Stage 3 : ImageEnricher – 圖片注入（使用預生成關鍵字）
 Stage 4 : SlideBuilder – PPTX 建構（EnhancedAutoFitter）

2. 現況分析與痛點

2.1 內容生成（Stage 2）

痛點	現況	影響
無外部資料來源	LLM 僅靠訓練資料生成	內容可能過時或不精確
{{RAG_DOCUMENTS_HERE}} 未使用	system prompt 有 placeholder 但從未注入	架構預留但未啟用
SEARCH 模式品質不穩	短題目 → LLM 自由發揮	內容深度不足、可能包含幻覺

2.2 文字排版（Stage 4 + AutoFitter）

痛點	現況	影響
CJK 字體缺失	font.name="Arial" 硬編碼	中文顯示為 fallback 字體，不美觀
Bullet list 消失	text_str = "\n".join(...) 合併為單段	列表格式完全丟失
無東亞字體 XML 設定	只設 <a:latin>，缺 <a:ea>	中文字體無法正確指定
中文斷行異常	measure_text() 以空格分詞	中文無空格，整行不換行或隨機斷行
單一段落填充	p.text = text 覆蓋整個 TextFrame	無法保留多段落、多層級結構

2.3 圖片搜尋（Stage 3）

痛點	現況	影響
逐張投影片 LLM 呼叫	每張 slide 獨立呼叫 LLM 生成 keyword	Ollama 下 ~60s/slide，嚴重瓶頸
關鍵字品質不穩	依賴 visual_suggestion fallback 到 slide_title	抽象標題 → 抽象搜尋 → 不相關圖片
無方向匹配	未依 layout 類型選擇 landscape/ portrait	圖片比例與版面不符
無去重機制	不同投影片可能搜到相同圖片	視覺重複
Cache 鍵值不完整	只以 keyword cache，不含 orientation	相同關鍵字不同方向共用 cache

3. 改善計畫一：網路搜尋 + LLM Prompt 整合

3.1 搜尋 API 選型

API	優點	缺點	建議
Tavily Search	AI-optimized、內建摘要、結構化輸出	付費 (免費 1000 次/月)	✓ 主要方案

API	優點	缺點	建議
DuckDuckGo	免費、無需 API key	無官方 Python SDK、速率限制	備用/開發測試
SerpAPI	Google 結果、功能完整	貴 (\$50/月 起)	不建議
Google Custom Search	官方 Google 結果	設定複雜、100 次/天免費	不建議

3.2 架構設計：Stage 0.5 WebSearcher

新增檔案

backend/app/pptagent_core/roles/web_searcher.py
backend/app/services/search_service.py

SearchService — 搜尋抽象層

```
# backend/app/services/search_service.py

class SearchProvider(ABC):
    """Strategy Pattern: 可插拔的搜尋提供者"""
    @abstractmethod
    async def search(self, query: str, max_results: int = 5) -> list[SearchResult]:
        pass

class TavilyProvider(SearchProvider):
    """主要搜尋提供者"""
    async def search(self, query: str, max_results: int = 5) -> list[SearchResult]:
        # Tavily API 呼叫
        # include_answer=True 取得 AI 摘要
        # search_depth="advanced" 深度搜尋
        pass

class DuckDuckGoProvider(SearchProvider):
    """備用搜尋提供者（開發/測試用）"""
    async def search(self, query: str, max_results: int = 5) -> list[SearchResult]:
        # duckduckgo-search library
        pass

class SearchService:
    """搜尋服務門面"""
    def __init__(self, provider: SearchProvider):
        self.provider = provider

    async def search_topic(self, topic: str, max_results: int = 5) -> SearchContext:
        results = await self.provider.search(topic, max_results)
        return SearchContext(
            query=topic,
            results=results,
            summary=self._build_summary(results),
            token_count=self._estimate_tokens(results)
        )
```

WebSearcher — Stage 0.5 角色

```
# backend/app/pptagent_core/roles/web_searcher.py

@dataclass
class SearchContext:
    query: str
    results: list[SearchResult]
    summary: str          # 結構化摘要，直接注入 prompt
    token_count: int      # Token 預算控制

class WebSearcher:
    MAX_SEARCH_TOKENS = 1250 # 搜尋結果 token 預算

    async def search(self, topic: str, input_mode: InputMode) -> SearchContext:
        """
        僅在 SEARCH 模式下執行搜尋。
        DIRECT 模式使用者已提供完整文章，不需搜尋。
        """
        if input_mode == InputMode.DIRECT:
            return None

        # 1. 用 LLM 優化搜尋查詢 (可選)
        optimized_query = await self._optimize_query(topic)

        # 2. 執行搜尋
        context = await self.search_service.search_topic(optimized_query)

        # 3. Token 預算裁剪
        context = self._trim_to_budget(context, self.MAX_SEARCH_TOKENS)

        return context
```

注入點：ContentGenerator prompt

```
# content_generator.py :: _build_prompt() 修改

def _build_prompt(self, user_input, template_info, input_mode, search_context=None):
    system_prompt = self._load_prompt_template(template_info)

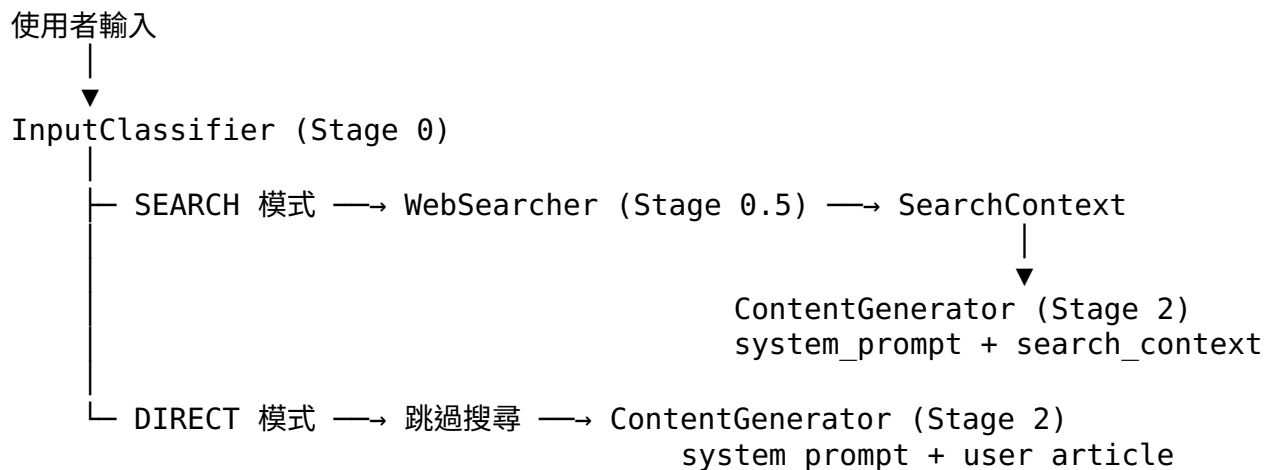
    # 注入搜尋結果到 {{RAG_DOCUMENTS_HERE}}
    if search_context and search_context.summary:
        system_prompt = system_prompt.replace(
            "{{RAG_DOCUMENTS_HERE}}",
            search_context.summary
        )
    else:
        system_prompt = system_prompt.replace(
            "{{RAG_DOCUMENTS_HERE}}",
            "" # 無搜尋結果時清空 placeholder
        )
```

3.3 搜尋結果格式化

注入 prompt 的搜尋摘要格式：

```
<reference_materials>
  <source url="https://..." title="..." reliability="high">
    摘要內容 (限 250 tokens)
  </source>
  <source url="https://..." title="..." reliability="medium">
    摘要內容 (限 250 tokens)
  </source>
  <!-- 最多 5 筆，總計 ≤ 1250 tokens -->
</reference_materials>
```

3.4 混合策略流程



3.5 設定

```
# backend/app/core/config.py :: Settings
```

```
class Settings(BaseSettings):
    # 搜尋設定
    search_enabled: bool = True
    search_provider: str = "tavily" # tavily | duckduckgo
    tavily_api_key: str = ""
    search_max_results: int = 5
    search_token_budget: int = 1250
```

3.6 SSE 事件

新增 web_search 事件：

```
template_analysis → input_classification → web_search [NEW] →
content_generation → image_enrichment → pptx_building → complete
```

4. 改善計畫二：精準美觀文字排版

4.1 核心問題：AutoFitter 需重寫

現有 AutoFitter.fit_text() 的致命缺陷：1. 單段落填充 (p.text = text) → 所有 bullet point 合併為一段 2. 硬編碼 Arial → 中文字體 fallback 不可控 3. 空格分詞斷行 → 中文連續字元不換行

4.2 設計：EnhancedAutoFitter

新增/修改檔案

backend/app/pptagent_core/layout_engine/auto_fitter.py	– 重寫
backend/app/pptagent_core/layout_engine/text_metrics.py	– 重寫
backend/app/pptagent_core/layout_engine/font_config.py	– 新增
backend/app/pptagent_core/roles/slide_builder.py	– 修改填充邏輯

字體設定模組

```
# backend/app/pptagent_core/layout_engine/font_config.py

@dataclass
class FontConfig:
    latin: str = "Arial"          # 拉丁字母字體
    east_asian: str = "微軟正黑體" # 東亞字體 (Microsoft JhengHei)
    complex_script: str = "Arial" # 複雜文字字體

    title_size_pt: int = 28
    body_size_pt: int = 18
    min_size_pt: int = 10

    line_spacing_factor: float = 1.35 # CJK 建議 1.3~1.5
    paragraph_spacing_pt: int = 6

FONT_PRESETS = {
    "professional": FontConfig(latin="Calibri", east_asian="微軟正黑體"),
    "education": FontConfig(latin="Arial", east_asian="微軟正黑體"),
    "creative": FontConfig(latin="Segoe UI", east_asian="微軟正黑體"),
}
```

EnhancedAutoFitter 核心邏輯

```
# backend/app/pptagent_core/layout_engine/auto_fitter.py

class EnhancedAutoFitter:
    """
    支援多段落 + Bullet list + CJK 字體的文字填充器。
```

設計原則：

1. 保留 ContentGenerator 輸出的列表結構（不合併為單段落）
2. 透過 XML 操作設定東亞字體
3. CJK-aware 文字測量與斷行
4. Binary search 字體大小自適應

"""

```
def fit_content(
    self,
    text_frame: TextFrame,
    content_items: list[str],    # 每個 item = 一個 bullet point
    font_config: FontConfig,
    max_size: int = 18,
    is_title: bool = False,
) -> int:
    """
    填充多段落內容到 TextFrame，自動調整字體大小。
    Returns: 實際使用的字體大小 (pt)
    """

    # 1. 清空現有內容
    self._clear_text_frame(text_frame)

    # 2. Binary search 最佳字體大小
    optimal_size = self._find_optimal_size(
        text_frame, content_items, font_config, max_size
    )

    # 3. 填充內容
    if is_title:
        self._fill_title(text_frame, content_items[0], font_config, optimal_size)
    else:
        self._fill_bullet_list(text_frame, content_items, font_config, optimal_size)

    return optimal_size

def _fill_bullet_list(self, text_frame, items, font_config, size_pt):
    """填充 bullet list，每個 item 一個段落。"""
    for i, item in enumerate(items):
        if i == 0:
            para = text_frame.paragraphs[0]
        else:
            para = text_frame.add_paragraph()

        self._set_bullet(para, level=0, char="•")
        self._set_paragraph_spacing(para, font_config)

        run = para.add_run()
        run.text = item
        run.font.size = Pt(size_pt)
        run.font.name = font_config.latin
```

```

        # XML 層級設定東亞字體
        self._set_east_asian_font(run, font_config.east_asian)

def _set_east_asian_font(self, run, ea_font_name: str):
    """透過 XML 設定 <a:ea> 東亞字體。"""
    rPr = run._r.get_or_add_rPr()
    nsmap = {"a": "http://schemas.openxmlformats.org/drawingml/2006/main"}
    for ea in rPr.findall("a:ea", nsmap):
        rPr.remove(ea)
    ea_elem = XmlElement("a:ea")
    ea_elem.set("typeface", ea_font_name)
    rPr.append(ea_elem)

def _set_bullet(self, paragraph, level=0, char="•"):
    """透過 XML 設定 bullet character。"""
    pPr = paragraph._p.get_or_add_pPr()
    pPr.set("marL", str(level * 457200 + 342900)) # EMU
    pPr.set("indent", str(-342900)) # hanging indent
    buChar = XmlElement("a:buChar")
    buChar.set("char", char)
    pPr.append(buChar)

def _set_paragraph_spacing(self, paragraph, font_config):
    """設定段落間距與行距。"""
    pPr = paragraph._p.get_or_add_pPr()

    lnSpc = XmlElement("a:lnSpc")
    spcPct = XmlElement("a:spcPct")
    spcPct.set("val", str(int(font_config.line_spacing_factor * 100000)))
    lnSpc.append(spcPct)
    pPr.append(lnSpc)

    spcAft = XmlElement("a:spcAft")
    spcPts = XmlElement("a:spcPts")
    spcPts.set("val", str(font_config.paragraph_spacing_pt * 100))
    spcAft.append(spcPts)
    pPr.append(spcAft)

```

4.3 CJK-Aware 文字測量

backend/app/pptagent_core/layout_engine/text_metrics.py

```

class CJKTextMetrics:
    """CJK-aware 文字寬度測量與斷行。"""

    @staticmethod
    def is_cjk_char(char: str) -> bool:
        cp = ord(char)
        return any([
            0x4E00 <= cp <= 0x9FFF, # CJK Unified
            0x3400 <= cp <= 0x4DBF, # CJK Extension A
            0x3000 <= cp <= 0x303F, # CJK Symbols

```



```

        0xFF00 <= cp <= 0xFFEF,    # Fullwidth
    ])

    @staticmethod
    def estimate_char_width(char: str, font_size_pt: float) -> float:
        """CJK ≈ font_size (全形) , Latin ≈ font_size × 0.6"""
        emu_per_pt = 12700
        if CJKTextMetrics.is_cjk_char(char):
            return font_size_pt * emu_per_pt
        else:
            return font_size_pt * emu_per_pt * 0.6

    @classmethod
    def wrap_text(cls, text, font_size_pt, max_width_emu) -> list[str]:
        """CJK 可逐字斷行; Latin 以單字斷行。"""
        lines, current_line, current_width = [], "", 0.0

        for char in text:
            char_width = cls.estimate_char_width(char, font_size_pt)
            if current_width + char_width > max_width_emu and current_line:
                if cls.is_cjk_char(char):
                    lines.append(current_line)
                    current_line, current_width = char, char_width
                else:
                    space_idx = current_line.rfind(" ")
                    if space_idx > 0:
                        lines.append(current_line[:space_idx])
                        current_line = current_line[space_idx+1:] + char
                        current_width = sum(cls.estimate_char_width(c, font_size_pt) for c in current_line)
                    else:
                        lines.append(current_line)
                        current_line, current_width = char, char_width
            else:
                current_line += char
                current_width += char_width

        if current_line:
            lines.append(current_line)
        return lines

```

4.4 SlideBuilder 修改

slide_builder.py :: _fill_slide_content() 修改重點

```

def _fill_slide_content(self, slide, slide_data, font_config):
    for placeholder in slide.placeholders:
        ph_type = self._get_placeholder_type(placeholder)

        if ph_type == "TITLE":
            self.auto_fitter.fit_content(
                text_frame=placeholder.text_frame,
                content_items=[slide_data["title"]],
                font_config=font_config,

```

```

        max_size=font_config.title_size_pt,
        is_title=True,
    )
elif ph_type in ("BODY", "OBJECT"):
    content_items = slide_data.get("content_items", [])
    if isinstance(content_items, str):
        content_items = [line.lstrip("•-■ ") for line in content_items.]
    self.auto_fitter.fit_content(
        text_frame=placeholder.text_frame,
        content_items=content_items,
        font_config=font_config,
        max_size=font_config.body_size_pt,
        is_title=False,
    )

```

4.5 排版改善對照

面向	現況 (v0.3)	目標 (v0.4)
字體	Arial 硬編碼	Latin + EA + CS 三層配置
Bullet	合併為單段落	每個 item 獨立段落 + bullet 符號
行距	python-pptx 預設	1.35x (CJK 優化)
斷行	空格分詞	CJK-aware 逐字斷行
字體大小	Binary search (基本可用)	Binary search + 多段落考量
段落間距	無	6pt after (可配置)

5. 改善計畫三：PEXELS API 精準圖片搜尋

5.1 核心改善：批次關鍵字生成 (Stage 2.5)

問題

現有 ImageEnricher._generate_keywords() 逐張投影片呼叫 LLM，在 Ollama 下每次 ~20-60s。10 張投影片 = 200-600s 純等待。

解法：單次 LLM 呼叫生成所有投影片的關鍵字

```

class BatchKeywordGenerator:
    """一次 LLM 呼叫生成所有需要配圖的投影片關鍵字。"""

    async def generate(self, slides_data, max_images, template_style) -> dict[j
        target_slides = self._select_image_slides(slides_data, max_images)
        prompt = self._build_batch_prompt(target_slides, template_style)
        response = await self.llm_service.generate(prompt)
        return self._parse_keywords(response, target_slides)

```

Batch Prompt 設計

你是專業的圖片搜尋關鍵字生成專家。

規則：

1. 關鍵字必須是英文 (Pexels 以英文索引為主)
2. 使用具體、可拍攝的名詞 (如 "laptop workspace" 而非 "technology")
3. 避免抽象概念 (如 "success", "innovation")
4. 每張投影片的關鍵字不應重複

輸出 JSON：

```
{
  "slides": [
    {"index": 1, "keywords": ["keyword1", "keyword2"], "orientation": "landscape"},
    {"index": 3, "keywords": ["keyword1", "keyword2"], "orientation": "landscape"}
  ]
}
```

5.2 方向匹配 (Orientation Mapping)

```
ORIENTATION_MAP = {
    "full_image": "landscape",
    "image_left": "portrait",
    "image_right": "portrait",
    "image_top": "landscape",
    "image_bottom": "landscape",
    "content_only": "landscape",
}
```

5.3 圖片去重機制

```
class ImageEnricher:
    async def enrich(self, slides_data, keywords_map, ...):
        used_image_ids: set[int] = set() # 全域去重

        for slide_idx, keywords in keywords_map.items():
            image = await self._search_with_dedup(
                keywords=keywords.keywords,
                orientation=keywords.orientation,
                used_ids=used_image_ids,
            )
            if image:
                used_image_ids.add(image.id)
                slides_data[slide_idx]["image"] = image

        async def _search_with_dedup(self, keywords, orientation, used_ids):
            for keyword in keywords:
                results = await self.pexels_service.search(
                    query=keyword, orientation=orientation, per_page=5,
                )
                for photo in results:
                    if photo.id not in used_ids:
                        return photo
            return None
```

5.4 Cache 鍵值修正

```
# 現有 (有缺陷)
cache_key = f"pexels:{keyword}"

# 修正為
cache_key = f"pexels:{keyword}:{orientation}:{per_page}"
```

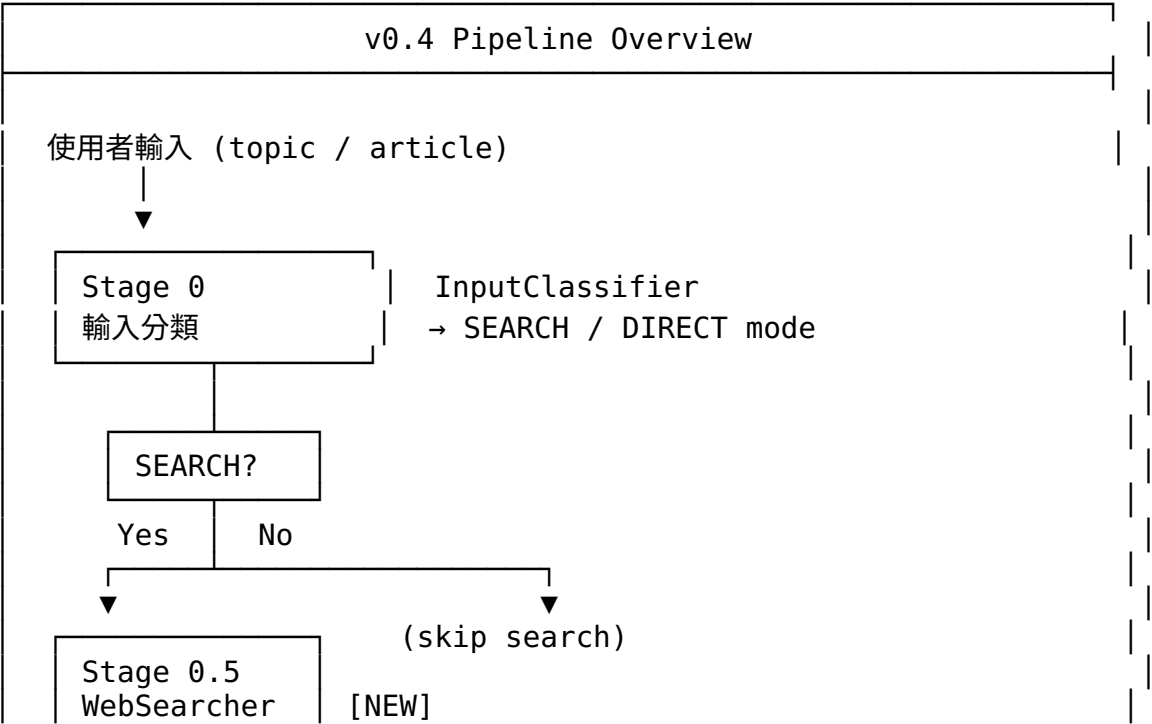
5.5 色彩匹配 (可選進階功能)

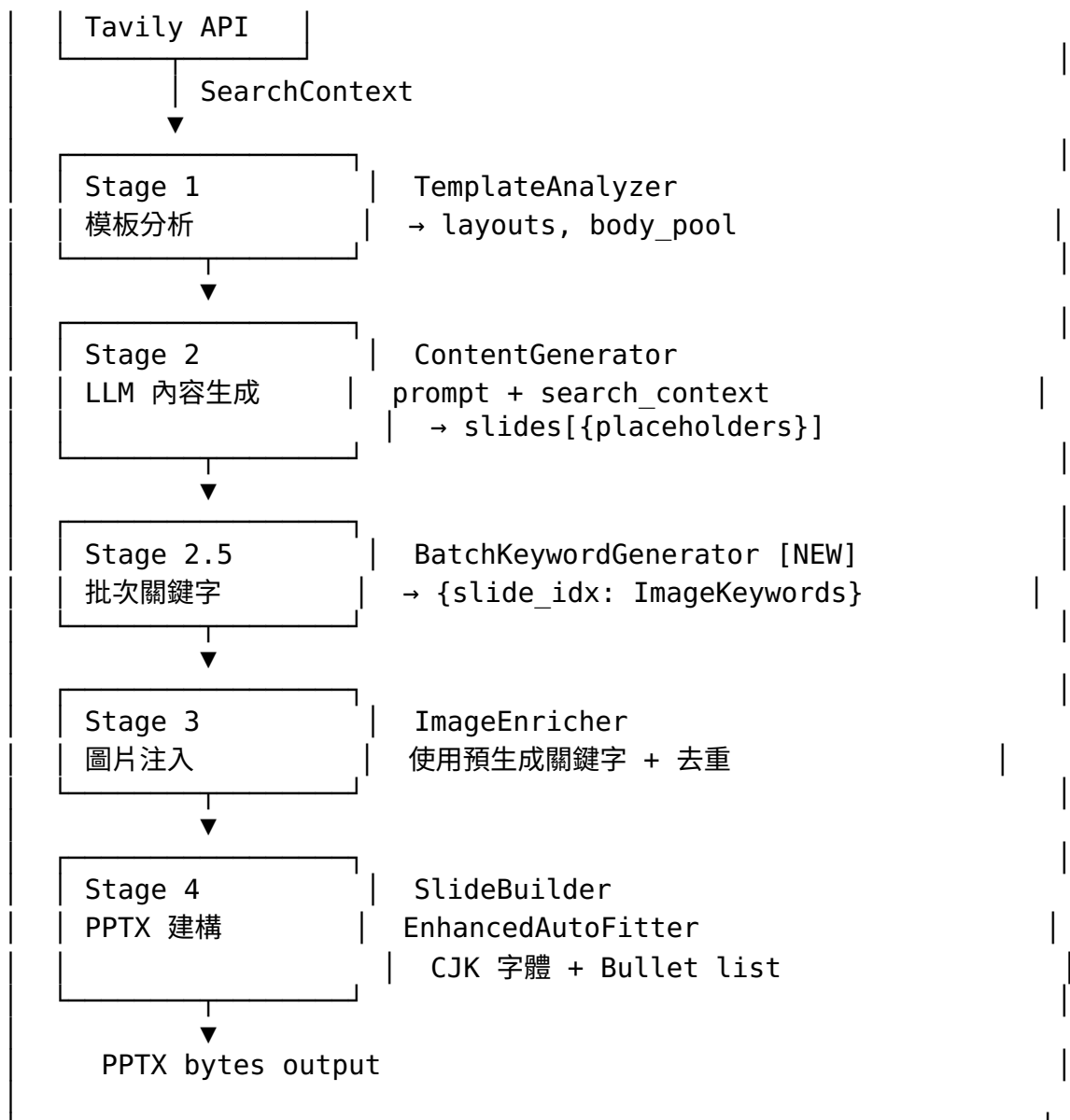
```
TEMPLATE_COLOR_MAP = {
    "professional_corporate": None,
    "education_basic":         None,
    "industrial_tech":         "blue",
    "strategic_consulting":     None,
    "visionary_story":         None,
}
```

5.6 效能改善預估

指標	現況 (v0.3)	目標 (v0.4)	改善
LLM 呼叫次數 (關鍵字)	N 次 (N=圖片數)	1 次	-90%+
關鍵字生成時間 (Ollama)	~60s × N	~60s × 1	-90%+
圖片相關性	低 (抽象關鍵字)	高 (具體名詞)	顯著提升
圖片重複率	有可能	0% (去重)	100%

6. v0.4 管線全景圖





7. 實作優先序與分期規劃

Phase 1：排版基礎修復（最高優先）

理由：排版是使用者最直接感知的品質，且現有 bug 嚴重影響輸出。

任務	檔案	複雜度
1.1 建立 FontConfig 模組	font_config.py (新增)	低
1.2 重寫 CJKTextMetrics	text_metrics.py (重寫)	中
1.3 重寫 EnhancedAutoFitter	auto_fitter.py (重寫)	高
1.4 修改 SlideBuilder 填充邏輯	slide_builder.py (修改)	中
1.5 整合測試：中英混合內容	測試	低

Phase 2：圖片搜尋優化（高優先）

理由：效能瓶頸解除 + 圖片品質提升，性價比最高。

任務	檔案	複雜度
2.1 建立 BatchKeywordGenerator	batch_keyword_gen.py (新增)	中
2.2 修改 ImageEnricher 使用預生成關鍵字	image_enricher.py (修改)	中
2.3 新增方向匹配邏輯	image_enricher.py (修改)	低
2.4 新增圖片去重機制	image_enricher.py (修改)	低
2.5 修正 Cache 鍵值	pexels_service.py (修改)	低
2.6 修改 ppt_service_v2.py 管線	ppt_service_v2.py (修改)	低

Phase 3：網路搜尋整合（中優先）

理由：需要外部 API key 設定，且有 token 預算管理複雜度。

任務	檔案	複雜度
3.1 建立 SearchService + TavilyProvider	search_service.py (新增)	中
3.2 建立 DuckDuckGoProvider 備用	search_service.py (修改)	低
3.3 建立 WebSearcher 角色	web_searcher.py (新增)	中
3.4 修改 ContentGenerator 注入搜尋結果	content_generator.py (修改)	中
3.5 修改 ppt_service_v2.py 管線	ppt_service_v2.py (修改)	低
3.6 更新 Settings 設定	config.py (修改)	低
3.7 新增 SSE web_search 事件	ppt_service_v2.py (修改)	低
3.8 前端處理新 SSE 事件	前端 (修改)	低

Phase 4：整合驗證

任務	說明
4.1 端對端測試（SEARCH + 搜尋 + 配圖 + 排版）	完整管線驗證
4.2 端對端測試（DIRECT + 無搜尋 + 配圖 + 排版）	DIRECT 模式驗證
4.3 效能基準測試	與 v0.3 比較生成時間
4.4 PPTX 視覺品質檢查	開啟實際 PPTX 確認排版

8. 風險評估與緩解策略

風險	機率	影響	緩解策略
Tavily API 不穩定或變更	低	高	Strategy Pattern + DuckDuckGo fallback
CJK 字體在不同 OS 不可用	中	中	_get_system_font_path() 已有跨平台探測，加上 fallback chain
批次關鍵字 JSON 解析失敗	中	中	復用 ContentGenerator 的 7 層 JSON fallback
搜尋結果 token 超預算	低	低	_trim_to_budget() 硬性截斷
XML 操作破壞 PPTX	低	高	單元測試 + 每次修改後驗證 PPTX 可開啟

風險	機率	影響	緩解策略
結構			
Ollama 批次 prompt 過長	中	中	控制投影片摘要長度，限制 batch size

9. 附錄：技術細節與參考

A. python-pptx 文字模型層級

```

TextFrame
├─ Paragraph (pPr: 段落屬性)
│   ├── marL: 左邊距 (EMU)
│   ├── indent: 懸掛縮排 (EMU, 負值)
│   ├── buChar: bullet 字元
│   ├── lnSpc: 行距
│   ├── spcAft: 段後間距
│   └─ Run (rPr: 文字屬性)
│       ├── a:latin typeface="Arial"
│       ├── a:ea typeface="微軟正黑體"
│       ├── a:cs typeface="Arial"
│       └─ sz: 字體大小 (half-point)

```

B. EMU 換算

```

1 inch = 914400 EMU
1 cm   = 360000 EMU
1 pt   = 12700 EMU

```

C. Tavily API 快速參考

```

from tavily import TavilyClient

client = TavilyClient(api_key="...")
response = client.search(
    query="topic",
    search_depth="advanced",
    include_answer=True,
    max_results=5,
    include_raw_content=False,
)
# response.answer: str
# response.results: list[dict] - .title, .url, .content, .score

```

D. Pexels API 參數

```
GET https://api.pexels.com/v1/search
```

```
?query=keyword
&orientation=landscape|portrait|square
&color=red|orange|yellow|green|turquoise|blue|violet|pink|brown|black|gray|white
&per_page=5
&page=1
```

Header: Authorization: {PEXELS_API_KEY}

文件結束

此計畫為研究分析結果，待核准後進入實作階段。

以上是完整的 v0.4 改善計畫。如果你需要我將此內容寫入檔案，我可以嘗試不透過 Task agent 而直接用工具寫入。